```
import uuid
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordRequestForm
from pydantic import BaseModel

from database import tenant_db, get_connection
from auth import hash_password, verify_password, create_access_token, get_current
from ai_summary import generate_kpi_summary

app = FastAPI(title="SaaS Analytics API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)


# ---------- Schemas ----------
class SignupRequest(BaseModel):
    tenant_name: str
    email: str
    password: str


class MetricIn(BaseModel):
    metric_name: str
    metric_value: float


# ---------- Auth routes ----------
@app.post("/auth/signup")
def signup(payload: SignupRequest):
    """
    Creates a brand new tenant + first admin user.
    Note: this runs OUTSIDE tenant_db() because at signup time
    there is no tenant yet -- we're creating one.
    """
    conn = get_connection()
    try:
        with conn.cursor() as cur:
```

```
            cur.execute(
                "INSERT INTO tenants (name) VALUES (%s) RETURNING id",
                (payload.tenant_name,),
            )
            tenant_id = cur.fetchone()["id"]

            hashed = hash_password(payload.password)
            cur.execute(
                """INSERT INTO users (tenant_id, email, hashed_password, role)
                   VALUES (%s, %s, %s, 'admin') RETURNING id""",
                (tenant_id, payload.email, hashed),
            )
            user_id = cur.fetchone()["id"]
        conn.commit()
    finally:
        conn.close()

    token = create_access_token(str(user_id), str(tenant_id), "admin")
    return {"access_token": token, "token_type": "bearer"}


@app.post("/auth/login")
def login(form_data: OAuth2PasswordRequestForm = Depends()):
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM users WHERE email = %s", (form_data.userna
            user = cur.fetchone()
    finally:
        conn.close()

    if not user or not verify_password(form_data.password, user["hashed_password"
        raise HTTPException(status_code=401, detail="Invalid email or password")

    token = create_access_token(str(user["id"]), str(user["tenant_id"]), user["ro
    return {"access_token": token, "token_type": "bearer"}


# ---------- Metrics routes (tenant-isolated) ----------
@app.post("/metrics")
def add_metric(payload: MetricIn, current_user: dict = Depends(get_current_user))
    with tenant_db(current_user["tenant_id"]) as conn:
        with conn.cursor() as cur:
            cur.execute(
                """INSERT INTO metrics (tenant_id, metric_name, metric_value)
                   VALUES (%s, %s, %s) RETURNING id""",
```

```
                (current_user["tenant_id"], payload.metric_name, payload.metric_v
            )
            metric_id = cur.fetchone()["id"]
    return {"id": metric_id, "status": "created"}


@app.get("/metrics")
def list_metrics(current_user: dict = Depends(get_current_user)):
    with tenant_db(current_user["tenant_id"]) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM metrics WHERE tenant_id = %s ORDER BY recorded_at
                (current_user["tenant_id"],),
            )
            rows = cur.fetchall()
    return rows


# ---------- AI summary route ----------
@app.post("/summary/generate")
def generate_summary(current_user: dict = Depends(get_current_user)):
    with tenant_db(current_user["tenant_id"]) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM metrics WHERE tenant_id = %s ORDER BY recorded_at
                (current_user["tenant_id"],),
            )
            metrics = cur.fetchall()

        summary_text = generate_kpi_summary(metrics)

        with conn.cursor() as cur:
            cur.execute(
                """INSERT INTO ai_summaries (tenant_id, summary_text)
                   VALUES (%s, %s) RETURNING id, generated_at""",
                (current_user["tenant_id"], summary_text),
            )
            result = cur.fetchone()

    return {
        "id": result["id"],
        "summary": summary_text,
        "generated_at": result["generated_at"],
    }
```

```
@app.get("/health")
def health():
    return {"status": "ok"}
```